Protección

- Protección
  - Actualizaciones
  - Logs
  - Autenticación
  - validación y sanitización
  - Cifrado Https
  - Variables de entorno
  - Inyección de SQL
  - Ataques de peticiones
    - Protección como middleware
    - Protegiendo una ruta
  - Establecer Límites
    - Limitar el tamaño del Json
    - Limitar la carga de un urlencoded
    - Limites en multer
      - Limitar el tamaño de un archivo
      - Cantidad de archivos
      - Cantidad de campos
      - Tamaño de los campos
  - Middleware Helmet
  - CORS

Actualizaciones

De ser posible usar las últimas versiones de los lenguajes, frameworks y librerías.
O de lo contrario estar al tanto de actualizaciones y parches de seguridad.

Logs

- Winston
- Morgan

Autenticación

- Basic Auth
- Digest Auth
- Bearer Token
- Json Web Token
- OAuth

validación y sanitización

- Express Validator
- Joi
- Zod

Cifrado Https

- Generar Certificado SSL
- Implementar Https en el servidor

Variables de entorno

- dotEnv

Inyección de SQL

Nunca construir consultas sql concatenando datos que envió el usuario.
Todas las consultas deben estar parametrizadas.

```
app.get('/usuarios', async (req, res) => {
  const { id } = req.id_usuario;

  //Mala práctica: Consulta concatenada
  const sql = `SELECT * FROM usuarios WHERE id = '${id}'`;

  try {
    const result = await pool.query(sql);
    res.json(result.rows);
  } catch (err) {
    res..json(err);
  }

});
```

```
app.get('/usuarios', async (req, res) => {
  const { id } = req.id_usuario;

  try {
    // recomendación: Consulta parametrizada
    const result = await pool.query(
      'SELECT * FROM usuarios WHERE id = $1',[id]
    );
    res.json(result.rows);
  } catch (err) {
    console.error(err);
    res.status(500);
    res.json({ error: 'Error interno del servidor' });
  }
});
```

Si el cliente hace esta petición `GET /usuarios?id=' OR '1'='1`

Concatenando campos para construir la sentencia sql, esta quedaría:
`SELECT id, nombre, email FROM usuarios WHERE email = '' OR '1'='1'`

Con una consulta parametrizada. la consulta no cambia y el parámetro se toma como literal `' OR '1'='1'`
Consulta: `SELECT * FROM usuarios WHERE id = $1`
Parámetro: `$1` = `"' OR '1'='1"`

Ataques de peticiones

Para proteger nuestra API REST de ataques de peticiones
podemos hacer uso del middleware express-rate-limit

Protección como middleware

```
const rateLimit = require('express-rate-limit');

const limiter = rateLimit({
  windowMs: 15 * 60 * 1000, // 15 minutos
  max: 100, // máximo 100 peticiones por IP
  message: {
    error: 'Demasiadas solicitudes, inténtalo más tarde'
  }
});

app.use(limiter);
```

Protegiendo una ruta

```
app.use('/login', rateLimit({
           windowMs: 15 * 60 * 1000,
           max: 5
}));
```

Establecer Límites

Limitar el tamaño del Json

```
app.use(express.json({
  limit: '1mb'
}));
```

Limitar la carga de un urlencoded

```
app.use(express.urlencoded({
  extended: true,
  limit: '1mb'
}));
```

Limites en multer

El middleware que estemos usando para procesar solicitudes FormData proveerá diversas opciones para establecer límites, por ejemplo con con multer:

Limitar el tamaño de un archivo

```
const multer = require('multer');

const upload = multer({
  dest: 'uploads/',
  limits: {
    fileSize: 10 * 1024 * 1024      // 10 MB
  }
});
```

Cantidad de archivos

```
const upload = multer({
  dest: 'uploads/',
  limits: { files: 5 }
});
```

Cantidad de campos

```
const upload = multer({
  dest: 'uploads/',
  limits: { files: 5 }
});
```

Tamaño de los campos

```
const upload = multer({
  limits: {
    fieldSize: 1024 * 100 // 100 KB
  }
});
```

Middleware Helmet

Helmet es un middleware de seguridad para servidores Express
que protege nuestra API de un conjunto de ataques Http
Por ejemplo:

- Oculta información innecesario del servidor
- Evita MIME Sniffing

```
const helmet = require('helmet');

app.use(helmet());
```

CORS

CORS es una política de seguridad de los navegadores para proteger al usuario
El middleware CORS nis ayuda controlar quien puede consumir la API desde un navegador

```
const cors = require('cors');

app.use(cors({
  origin: [
    'https://app.miempresa.com'
  ]
}));
```